

## רמזים · המהדר קורא את הקוד שלך

שבוע: 12

## מטרה

רמז אחד לכל שורה שנבדקת, בסדר שבו תפגוש אותן. בכל אחד שתי דרגות: **דחיפה**, ואם לא הספיקה — **איפה הקלדת את זה היום**. הפתרון לא כאן; הוא במחזוריים, על ספריית ההשאלה, בשמות אחרים. ובשבוע הבא, במבחן, אין גם את זה.

## 1 · המפרט קיים ומלא

**דחיפה**. חמש כותרות, ומתחת לכל אחת משהו שאינו הערה. סימן `CODE HERE` שנשאר — גם בתוך הערה — הוא הסימן שהסעיף לא נכתב.

**איפה הקלדת את זה**. מחזור 4: שלוש השורות על הלוח לפני המהלך הראשון הן סעיף 4 בזעיר אנפין.

## 2 · המפרט נכתב לפני הקוד

**דחיפה**. `git log --oneline -- specs/ts-migration.md ts/` — איזה קומיט בא קודם? אם עוד לא עשית קומיט למפרט, עשה אותו **עכשיו**, לפני שאתה נוגע ב- `ts/`.

**איפה הקלדת את זה**. מחזור 3: כתבת את P2 לפני שקראת את ה-P2 שבקובץ. סעיף 5 הוא אותו הרגל — ארבעה דברים שתחפש בפלט, לפני שיש פלט.

3 · `ts/types.ts` מתאר את הישות

**דחיפה**. שמות השדות כבר קיימים — ב- `js/items.js`. איחוד אחד לפחות של ליטרלים: ארבע הסיבות לכישלון ש- `api.js` כבר זורק (חפש את `new ApiError`) וקרא את הארגומנט הראשון).

**איפה הקלדת את זה**. מחזור 1, צעד 1: `ShelfEntry` כ- `interface`, `LookupKind` כ- `type`.

## 4 · השומר מצמצם

**דחיפה**. טיפוס ההחזרה הוא `value is Item`, לא `boolean`. והגוף בודק שדות — לפחות שלושה, עם `typeof`. שומר שמחזיר `true` מתקמפל ולא שומר על כלום.

**איפה הקלדת את זה**. מחזור 1, צעד 4: `isEntry`. `isRecord` בשורה הראשונה, כי `typeof null === 'object'` ו- `typeof [] === 'object'` — ואז שדה אחרי שדה.

5 · אין `any`

**דחיפה**. `grep -rn "any" ts/`. כל שורה שחוזרת היא או הערה, או מקום שבו כיבית את המהדר. `catch (error)` הוא `unknown` תחת `strict` — והתשובה היא `error instanceof Error`, לא `any`.

**איפה ראית את זה.** מחזור 3, "אנחנו עושים": שני `any` הפכו ל-`unknown`, והמהדר הצביע על השורות של `payload` שצריכות שומר. את `catch` הוא לא סימן — את הצורה שלו תמצא בטבלה בבריק ( `error instanceof Error` לפני `error.name` ), וכבר כתובה ב- `js/api.js` שבתיקיית הפתיחה:

```
error instanceof Error ? error.message : String(error)
```

## 6 • השומר קפדן ומתירני במקומות הנכונים

**דחיפה.** מה `render` צריך כדי לא להישבר? את זה בודקים בקפדנות. מה מוצג רק בתנאי? את זה לא פוסלים.

**איפה הקלדת את זה.** ה-`you-do` של מחזור 1: `isBook` דחה ספר בלי שם, וקיבל ספר שהשנה שלו `null`.

## 7 • שכבת הרשת היא TypeScript עכשיו

**דחיפה.** אחרי `npx tsc -p tsconfig.json`, פתח את `js/api.js`: אין בו `interface`, אין `unknown` : , אין `import type`. אם יש — זה הקובץ הישן, והמהדר לא כתב (כנראה בגלל שגיאה; גלול למעלה בטרמינל).

**איפה הקלדת את זה.** מחזור 1, צעד 2: שתי שגיאות, ו-`js/` לא זז — `noEmitOnError`.

## 8 • unknown בגבול

**דחיפה.** `res.json()` מחזיר `Promise<any>` — מהספרייה, לא ממך. מילה אחת מחזירה את הגבול: המשתנה שמקבל את התשובה מוקלד `unknown`. `as` על התשובה הוא הבטחה, לא בדיקה.

**איפה הקלדת את זה.** מחזור 1: `let parsed: unknown` על `JSON.parse` — אותה בעיה בדיוק, על הדיסק במקום על הרשת. ומחזור 3: `as Book[]` החליף את השומר, והמסך אמר 7 כשהיו 6.

## 9 • מתקמפל תחת strict

**דחיפה.** תקן את השגיאה הראשונה בלבד והרץ שוב. שגיאה בטיפוס מוקדם מייצרת עשר במקומות שקוראים לו. ואם המספר **עלה** אחרי תיקון — זה בדרך כלל סימן טוב.

**איפה הקלדת את זה.** מחזור 1, צעדים 2-3: שתיים, ארבע, אפס. ואם כתוב

```
Property 'kind' does not exist on type 'ApiError'
```

— מחזור 3: השדה מוצהר בראש המחלקה.

## 10 • ההתנהגות זהה

**דחיפה.** העמוד ריק והמהדר עבר? חפש ב- `js/` ייבוא שנגמר בלי `js`. תשובה של 404 מופיעה כתוצאות? בדיקת `res.ok` נעלמה בדרך.

**איפה הקלדת את זה.** מחזור 1, צעד 4: `from './types'` — אפס שגיאות, עמוד ריק ו-404. ובכל מקרה:

```
git diff js/api.js
```

מול הקובץ שקיבלת — תנאי שהשתנה הוא התשובה.

## 11 • ההעברה קראה את הקוד

**דחיפה.** שלוש שאלות לעצמך: הסכמה של ה-API מוצהרת ב- `api.ts` ולא מיוצאת? לכל פונקציה מיוצאת יש טיפוס החזרה כתוב? ואתה יודע להסביר למה בחרת `interface` או `type`?

**איפה הקלדת את זה.** מחזור 1: `load(): Shelf` ו- `save(...): void` — טיפוס החזרה כתוב, גם כשהוא היה נגזר לבד.

## 12 · העוזר גנרי

**דחיפה.** `<T extends Element = Element>` – שלושה חלקים, וכל אחד עושה משהו. שתי פונקציות מיוצאות, ולפחות אחת מהן באמת קוראת ל-`querySelector`. החתימות שבשלד, עם `throw` ו-`return []`, הן עוד לא עוזר.

**איפה הקלדת את זה.** מחזור 2, צעדים 2-3: `$` ו-`inputText`. ה-`throw` עם הבורר הוא מה שמבדיל עוזר כן מ-`as` בשורה אחת.

## 13 · העוזר מחובר

**דחיפה.** קובץ JavaScript אחד לפחות מייבא מ-`./dom.js` – **הפלט**, לא `./dom.ts` – ומשתמש במה שייבא. אחרי החיבור, רענן: הקונסולה נקייה?

**איפה הקלדת את זה.** מחזור 2, צעד 4: `app.js` נשאר JavaScript ומייבא את `$`.

## 14 · הביקורת על מה שהמכונה כתבה

**דחיפה.** שני דברים שהכלי כתב ושינית – ולכל אחד: מה בדקת, ואיך. "הפלט היה בסדר" אינו ביקורת. אם הכלי לא החזיר אף אחד מהארבעה שבברייף – מחק כל השתקה והרץ שוב; ואם עדיין לא, כתוב את זה, עם מה שהרצת.

**איפה הקלדת את זה.** מחזור 3, ה-`you-do`: שורת הסקירה, בצורה של `ai-exemplars/exemplars/ai-review-js-good.md` בחומרי הקורס – הטענה והמסקנה בשתי שורות, וההוכחה היא משהו שרץ.

## אם נגמר הזמן

**חלקים א, ב ו-ג הם הליבה, והם 72 מתוך 100.** אם נשארו לך עשר דקות – סיים אותם, ותכתוב ב-`PROMPTS.md` את הביקורת (חלק ה, 12 נקודות) לפני שאתה מתחיל את חלק ד. **הביקורת לוקחת חמש דקות ושווה יותר מהגנריקה**, ובניגוד אליה אי אפשר להשלים אותה בבית מהזיכרון.